Structures conditionnelles et répétitives

Obj. : Comprendre et maîtriser les structures conditionnelles (**IF, CASE**) et les structures répétitives (**WHILE, REPEAT, LOOP**) dans MySQL.

Les structures de contrôle conditionnelles

Les structures conditionnelles permettent d'exécuter des instructions différentes selon si une condition est vraie ou fausse.

Instruction IF … THEN … END IF

✓ Syntaxe générale :

```
IF condition THEN
   instructions ;
END IF ;
```

Formateur : AARAB HAFID

✓ Exemple :

-- Vérifier si le téléphone est un portable

Requête :

```
 IF SUBSTR(v_telephone,1,2) = '06' THEN      -- SUBSTR extrait les deux premiers caractères
   SELECT 'C'est un portable';
ELSE                                          -- Sinon
   SELECT 'C'est un fixe ...';
END IF;                                       -- Fin du IF
```

On commence une condition IF qui vérifie si les deux premiers caractères de la variable v_telephone sont égaux à '06'.

Si la condition est vraie, MySQL affiche le texte 'C'est un portable'.

IF … ELSE … END IF

Requête :

```
IF condition THEN
   instructions ;
ELSE
   autres_instructions ;
END IF ;
```

Exemple :

```
DECLARE v_note DECIMAL(4,2) DEFAULT 12.5;   -- Déclare une variable v_note avec une valeur par défaut 12.5
IF v_note >= 16 THEN                        -- Vérifie si la note est supérieure ou égale à 16
   SELECT 'Mention Très Bien';              -- Si vrai, affiche 'Mention Très Bien'
ELSEIF v_note >= 14 THEN                    -- Sinon, vérifie si la note est supérieure ou égale à 14
   SELECT 'Mention Bien';                   -- Si vrai, affiche 'Mention Bien'
ELSEIF v_note >= 12 THEN                    -- Sinon, vérifie si la note est supérieure ou égale à 12
   SELECT 'Mention Assez Bien';             -- Si vrai, affiche 'Mention Assez Bien'
ELSE                                        -- Sinon
   SELECT 'Passable';                       -- Affiche 'Passable'
END IF;                                     -- Fin de la structure IF...ELSEIF
```

IF … ELSEIF … ELSE … END IF

Requête :

```
IF condition1 THEN
   instructions ;
ELSEIF condition2 THEN
   instructions ;
ELSE
   instructions ;
END IF ;
```

✓ Exemple :

Requête :

```
DELIMITER //          -- On change le délimiteur pour permettre l'utilisation de ';' dans le corps de la fonction
CREATE FUNCTION IncomeLevel(monthly_value INT) -- Création d'une fonction nommée IncomeLevel qui prend un paramètre entier monthly_value
RETURNS VARCHAR(20) -- La fonction retournera une chaîne de caractères de longueur maximale 20
BEGIN -- Début du bloc de la fonction
   DECLARE income_level VARCHAR(20);  -- Déclaration d'une variable locale pour stocker le résultat temporairement
```

```
    IF monthly_value <= 4000 THEN  -- Si la valeur mensuelle est inférieure ou égale à 4000
        SET income_level = 'Low Income';  -- On assigne la catégorie "Low Income" à la variable income_level
    ELSEIF monthly_value <= 7000 THEN  -- Si la valeur mensuelle est inférieure ou égale à 7000
        SET income_level = 'Avg Income';  -- On assigne la catégorie "Avg Income" à income_level
    ELSE -- Dans tous les autres cas (valeur > 7000)
        SET income_level = 'High Income';  -- On assigne la catégorie "High Income" à income_level
    END IF; -- Fin de la structure conditionnelle IF...ELSEIF...ELSE
    RETURN income_level; -- La fonction renvoie la valeur de income_level
END //  -- Fin du bloc de la fonction
DELIMITER ;          -- On revient au délimiteur par défaut ';'
```

Structure CASE

Il existe deux types de CASE :

✓ CASE avec sélecteur

✓ CASE sans sélecteur

CASE avec sélecteur

✓ Syntaxe :

```
CASE variable
    WHEN expr1 THEN instructions
    WHEN expr2 THEN instructions
    ...
    ELSE instructions
END CASE
```

✓ Exemple :

Requête :

```
DELIMITER $$ -- On change le délimiteur pour permettre l'utilisation de ';' dans le corps de la procédure
CREATE PROCEDURE GetCustomerShipping( -- Création d'une procédure nommée GetCustomerShipping
    IN pCustomerNumber INT,  -- Paramètre d'entrée : numéro du client
```

```
    OUT pShipping VARCHAR(50) -- Paramètre de sortie : mode de livraison
)
BEGIN  -- Début du bloc de la procédure
    DECLARE customerCountry VARCHAR(50); -- Déclaration d'une variable locale pour stocker le pays du client
    SELECT country INTO customerCountry  -- Récupère le pays du client
    FROM customers -- Depuis la table customers
    WHERE customerNumber = pCustomerNumber; -- Pour le client dont le numéro correspond à pCustomerNumber
    CASE customerCountry  -- Début d'une structure CASE sur la variable customerCountry
        WHEN 'USA' THEN  -- Si le pays est 'USA'
            SET pShipping = '2 day Shipping';    -- On assigne '2 day Shipping' à la variable de sortie
        WHEN 'Canada' THEN  -- Si le pays est 'Canada'
            SET pShipping = '3 day Shipping';    -- On assigne '3 day Shipping'
        ELSE
            SET pShipping = '5 day Shipping';    -- Tous les autres
    END CASE;
END $$
DELIMITER ;
```

CASE sans sélecteur

✓ Syntaxe :

```
CASE
   WHEN condition1 THEN instructions
   WHEN condition2 THEN instructions
   ELSE instructions
END
```

✓ Exemple dans SELECT :

Requête :

```
SELECT
   OrderID,
   Quantity,
   CASE
      WHEN Quantity > 30 THEN 'The quantity is greater than 30'
      WHEN Quantity = 30 THEN 'The quantity is 30'
      ELSE 'The quantity is under 30'
   END AS result -- Fin de la structure CASE et nommage du résultat sous l'alias "result"
FROM OrderDetails;
```

✓ Exemple avec variables :

Requête :

```
DECLARE v_mention CHAR(2); -- Déclare une variable locale nommée v_mention de type caractère, pouvant contenir 2 caractères

DECLARE v_note DECIMAL(4,2) DEFAULT 9.8; -- Déclare une variable locale nommée v_note de type décimal avec 4 chiffres dont 2 après la virgule, et initialise sa valeur à 9.8

CASE
   WHEN v_note >= 16 THEN SET v_mention = 'TB';  -- Très Bien
   WHEN v_note >= 14 THEN SET v_mention = 'B';   -- Bien
   WHEN v_note >= 12 THEN SET v_mention = 'AB'; -- Assez Bien
   WHEN v_note >= 10 THEN SET v_mention = 'P';   -- Passable
   ELSE SET v_mention = 'NP';                    -- Non Passable
END CASE;
```

Les structures répétitives (boucles)

Les boucles permettent de répéter une ou plusieurs instructions un certain nombre de fois.

Boucle LOOP

✓ Syntaxe :

```
label: LOOP
   instructions ;
END LOOP label;
```

✓ Exemple avec LEAVE (équivalent de break) :

Requête :

```
CREATE TABLE test(value INT);                    -- Crée une table nommée 'test' avec une colonne 'value' de type entier
DELIMITER $$                                     -- Change le délimiteur pour permettre l'utilisation de ';' dans le corps de la procédure
CREATE PROCEDURE ps_nombres()                    -- Création d'une procédure stockée nommée 'ps_nombres'
BEGIN                                            -- Début du bloc de la procédure
   DECLARE a INT DEFAULT 1;                      -- Déclare une variable locale 'a' initialisée à 1
   simple_loop: LOOP                             -- Début d'une boucle nommée 'simple_loop'
      INSERT INTO test VALUES(a);                -- Insère la valeur actuelle de 'a' dans la table 'test'
      SET a = a + 1;                             -- Incrémente la variable 'a' de 1
      IF a = 11 THEN                             -- Vérifie si 'a' a atteint 11
```

```
        LEAVE simple_loop;          -- Quitte la boucle si la condition est vraie
    END IF;                         -- Fin de la condition IF
  END LOOP simple_loop;             -- Fin de la boucle LOOP nommée 'simple_loop'
END $$                              -- Fin du bloc de la procédure
DELIMITER ;                         -- Retour au délimiteur par défaut ';'
```

✓ Exemple avec ITERATE (équivalent de continue)

Requête :

```
DELIMITER $$                        -- Changer le délimiteur pour permettre l'utilisation de ';' dans le corps de la procédure
CREATE PROCEDURE LoopDemo()         -- Création d'une procédure nommée 'LoopDemo'
BEGIN                               -- Début du bloc de la procédure
  DECLARE x INT DEFAULT 0;          -- Déclare une variable locale 'x' initialisée à 0
  DECLARE str VARCHAR(255) DEFAULT '';  -- Déclare une variable locale 'str' pour stocker la chaîne finale, initialisée vide
  loop_label: LOOP                  -- Début d'une boucle nommée 'loop_label'
    IF x > 10 THEN                  -- Vérifie si x est supérieur à 10
      LEAVE loop_label;             -- Quitte immédiatement la boucle si la condition est vraie
```

```
        END IF;
        SET x = x + 1;                    -- Incrémente x de 1
        IF (x MOD 2) THEN                 -- Vérifie si x est impair (modulo 2)
            ITERATE loop_label;           -- Ignore le reste de la boucle et passe à l'itération suivante
        ELSE
            SET str = CONCAT(str, x, ',');  -- Si x est pair, l'ajoute à la chaîne str avec une virgule
        END IF;
    END LOOP;                             -- Fin de la boucle LOOP
    SELECT str;                           -- Affiche le résultat final (tous les nombres pairs de 2 à 10 séparés par des virgules)
END $$                                    -- Fin du bloc de la procédure
DELIMITER ;                               -- Retour au délimiteur par défaut ';'
```

Boucle WHILE

✓ Exemple :

Requête :

```
CREATE PROCEDURE dowhile()                -- Création d'une procédure nommée 'dowhile'
BEGIN                                     -- Début du bloc de la procédure
```

```
    DECLARE v1 INT DEFAULT 5;          -- Déclare une variable locale 'v1' initialisée à 5
    WHILE v1 > 0 DO                    -- Début de la boucle WHILE : exécute le bloc tant que v1 > 0
        SET v1 = v1 - 1;               -- Décrémente la variable v1 de 1 à chaque itération
    END WHILE;                         -- Fin de la boucle WHILE
END;                                   -- Fin du bloc de la procédure
```

Explications :

- La boucle WHILE répète les instructions tant que la condition est vraie.
- Ici, la procédure décrémente v1 de 5 à 0.
- Comparée à LOOP, WHILE vérifie la condition **avant chaque itération** (la boucle peut ne jamais s'exécuter si la condition initiale est fausse).

Boucle REPEAT … UNTIL

✓ Exemple :

Requête :

```
DELIMITER //                           -- Changer le délimiteur pour permettre l'utilisation de ';' dans le corps de la procédure
```

```
CREATE PROCEDURE dorepeat(p1 INT)          -- Création d'une procédure nommée 'dorepeat' avec un paramètre d'entrée p1
BEGIN                                      -- Début du bloc de la procédure
   SET @x = 0;                             -- Initialise la variable de session @x à 0
   REPEAT                                  -- Début de la boucle REPEAT (exécute toujours au moins une fois)
      SET @x = @x + 1;                     -- Incrémente @x de 1 à chaque itération
   UNTIL @x > p1                           -- Condition de sortie : la boucle continue tant que @x <= p1
   END REPEAT;                             -- Fin de la boucle REPEAT
END //                                     -- Fin du bloc de la procédure
DELIMITER ;                                -- Retour au délimiteur par défaut ';'
```

Explications :

- La boucle **REPEAT** exécute toujours le bloc au moins une fois avant de tester la condition UNTIL.
- Ici, la boucle incrémente @x jusqu'à ce qu'elle dépasse la valeur passée en paramètre p1.
- Les variables préfixées par @ sont des **variables de session**, accessibles uniquement pour la session en cours.

En Résumé :

✓ **Les structures conditionnelles :**

- IF … THEN … END IF
- IF … ELSE … END IF
- IF … ELSEIF … ELSE … END IF
- CASE avec sélecteur
- CASE sans sélecteur

✓ **Les boucles :**

- LOOP + LEAVE (break) + ITERATE (continue)
- WHILE
- REPEAT … UNTIL

✓ **À retenir :**

- IF = conditions simples
- CASE = plusieurs choix
- LOOP = boucle la plus flexible
- WHILE = boucle “tant que”
- REPEAT = boucle exécutée au moins une fois

OFPPT

Formateur : AARAB HAFID